

IT3051 - FUNDAMENTALS OF DATA MINING

Final Report



BSc. (Hons) Information Technology Specializing in Data Science

Year 3 Semester 2, 2026

Student Name	Registration Number
Lokuhewage G L N D	IT23809710
Wathsara W H H	IT23858770
Irunika W N	IT23559004
Dulshini W G S	IT23720824

Table of Contents

1.	Introduction and Problem Definition	3
2.	Assigned Scenario and User/Stakeholder Requirements	5
3.	Dataset Identification, Source, Citation, and Validation	7
4.	Data Understanding and EDA	9
5.	Data Cleaning and Preprocessing	12
6.	Feature Engineering and Feature Selection	14
7.	Algorithms Implemented and Their Rationale	16
8.	Model Evaluation and Comparison	17
9.	Hyperparameter Tuning and Optimization	18
10.	Final Model Selection and Justification.....	20
11.	System Architecture and Implementation.....	21
12.	Backend and Frontend Development.....	24
	Backend Development	24
	Frontend Development.....	24
	End-to-End Prediction Flow	25
13.	System Testing and Results	26
14.	Limitations and Possible Future Improvements	29
	Possible Future Improvements.....	30
15.	Individual/Group Contributions, Where Required	31

1. Introduction and Problem Definition

Business Context

Delivering orders reliably presents a practical difficulty in a multi-seller marketplace, since an order can include various seller locations, different product features, diverse payment arrangements and different promised service periods. Once the order has been delivered to the customer after the promised date, the platform could receive complaints, have to deal with support requests, issue refunds and suffer a loss of customer confidence. By identifying potential problems before the item is dispatched, operational staff have the chance to examine the order while it is still possible to take action. The project does not attempt to estimate the savings from refunds since the dataset contains no intervention experiment or a reliable cost model. The suggested solution is an early-warning classifier for Olist orders. This classifier transforms the information about the order before dispatch into an estimated score for late delivery and a binary warning. The point at which the prediction is made is after payment has been approved and the item and seller have been confirmed, but before the carrier is handed the item. This interpretation is necessary since the feature set that has been implemented includes `approval_lag_hours`. To describe the current model as operating at the time of purchase would be inconsistent with its inputs. A version of the model based on purchase time would have to exclude the features that depend on approval and would need to be trained separately.

Objective, target and scope

The task involved binary classification, defined so that `is_late_delivery` equals 1 if `order_delivered_customer_date` is after `order_estimated_delivery_date`; in all other cases it is 0. Labels are created only in instances where the order has been delivered and there is a known actual delivery time stamp. For determining eligibility and assigning the labels, the outcome timestamps and the final order status are used, after which they are excluded from the predictors. The system is designed to determine whether a promised delivery time will be missed; it does not attempt to predict the exact arrival date, optimize routes, or forecast cancellation or non-delivery. The technical aims include constructing a validated order-level table, examining the distributions and identifying any quality problems, justifying the preprocessing and the use of engineered features, comparing at least four different algorithms, optimising the model settings and the warning threshold, and establishing a reproducible inference process. The practical aim is to prioritise the orders for human review. The model's success is evaluated using minority-class precision, recall, F1 score and average precision rather than relying solely on accuracy.

Target-definition sensitivity

Estimated delivery timestamps are recorded at midnight. Therefore, a delivery later on the same promised calendar date is classified as late under the strict timestamp rule. There are 1,292 such orders, representing 16.5% of the strict late group. A calendar-day definition would reduce the late rate from 8.11% to 6.77%. The current experiments retain the strict timestamp definition.

2. Assigned Scenario and User/Stakeholder Requirements

Scenario and decisions supported

Before dispatching an order, an operations user looks at the approved order. The application will get the details of the confirmed order, use the same features that were used during training, and then provide a risk estimate together with a warning. If an order is flagged, this could lead to a request for seller confirmation, a review of the dispatch, or proactive communication with the customer. The score must allow for proportionate human intervention; it must not automatically penalise a customer, a seller or a geographic area.

Stakeholder	Requirement	Decision supported
Operations/logistics staff	Identify orders needing attention before dispatch	Prioritize review and carrier or seller follow-up
Sellers and seller support	Understand which confirmed orders need monitoring	Check readiness, packaging and dispatch commitment
Customer support	Receive advance warning of possible promise failure	Prepare timely, transparent updates
Marketplace managers	Understand aggregate risk and alert workload	Allocate review capacity and monitor service patterns
Customers	Receive dependable communication	Make informed decisions when a delay is possible

Functional and non-functional requirements

The proposed application must accept order details through labelled inputs or a structured order import; validate types, required fields, ranges and date ordering; apply frozen preprocessing; and return a score, binary warning, model version and understandable recommendation. The saved binary rule is $p \geq 0.10$. The system provides Low, Medium and High risk labels using cutoffs of 0.30 and 0.55. The binary late-delivery alert uses a separate threshold of 0.10. These cutoffs support the prototype and require further validation against operational costs.

Usability requires clear labels, units, examples and readable errors. Reliability requires deterministic predictions for the same valid input and artifact version. Maintainability requires versioned feature definitions and preprocessing alongside the model. Privacy requires avoiding direct identifiers and precise personal addresses in the interface and logs. Performance should be measured against the proposal's goal of a response within a few seconds on the demonstration environment. These are acceptance requirements. Thirty repeated local API requests returned identical successful responses, with a median request time of 18.68 ms.

Inputs and outputs

Inputs should represent shipping commitment, purchase timing, customer and seller geographic context, item composition, product size/weight, order value, freight and approved payment structure. Derived values such as distance, item counts and ratios should be computed by the service from underlying records, not entered manually as sixty-seven independent fields. The proposed output combines the estimated score, warning status and an action such as routine handling or priority review. Because calibration has not been evaluated, the score should not be presented as a guaranteed real-world percentage of lateness.

3. Dataset Identification, Source, Citation, and Validation

Source and suitability

The dataset is the Brazilian E-Commerce Public Dataset by Olist, published by Olist on Kaggle. It describes anonymised Brazilian marketplace transactions from 2016-2018. The original source is <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>. The working citation is: Olist. (2018). Brazilian E-Commerce Public Dataset by Olist [Data set]. Kaggle. The supplied proposal records CC BY-NC-SA 4.0 as the licence; the final submission should retain the publisher's current licence and attribution record with the downloaded dataset.

The data is suitable because actual and estimated delivery timestamps support an explicit target, the order population is sufficiently large for chronological evaluation, and relational tables provide meaningful temporal, geographic and commercial predictors. Its complexity supports genuine data-mining work: target construction, aggregation, multi-table joins, missingness, imbalance and leakage control. It also has a clear limitation: an anonymised historical Brazilian sample cannot establish present-day performance in another country or marketplace.

Structure and analytical grain

Table/file group	Grain and role	Integration decision
Orders	One row per order; 99,441 raw orders	Defines target population and merge spine
Customers	Order-linked customer record	Provides destination geography
Sellers	One seller record; 3,095 sellers	Enrich sellers before order-level aggregation
Geolocation	Multiple coordinate observations per ZIP prefix	Deduplicate and form 19,015 ZIP lookups
Order items and products	Item-level transactions and product attributes	Aggregate confirmed items to order level
Category translation	Portuguese-to-English category lookup	Translate categories before grouping
Payments	One or more payment records per order	Aggregate to 99,440 order-level payment records
Reviews	Post-order review information	Excluded from predictors

The nine source CSV files form a relational dataset rather than a ready-made flat modelling table. Join keys include `order_id`, `customer_id`, `seller_id` and `product_id`. IDs are used to construct relationships but are removed from the model input. The final merge starts from 96,470 labelled orders and preserves that count through every join. Item and payment rows are aggregated before the merge, preventing orders with more items or payments from being duplicated.

Eligibility, Validation and Provenance

There are 96,478 delivered raw orders; eight have no known delivery timestamp, leaving 96,470 eligible labels. A further 2,963 orders have other statuses. Together, 2,971 raw orders remain outside the labelled modelling population. These are unlabelled for this task, rather than negative examples. The model consequently estimates lateness conditional on the historical delivered-order population and does not establish risk for cancelled or unavailable orders.

4. Data Understanding and EDA

Target imbalance and temporal behaviour

The labelled population contains 88,644 on-time and 7,826 late orders: 91.8876% and 8.1124%, respectively, an 11.33:1 ratio. A classifier predicting every order as on time would achieve 91.89% population accuracy while finding no late orders. This explains the choice of minority-class metrics and threshold optimization. Figure 1 visualizes the actual labelled population.

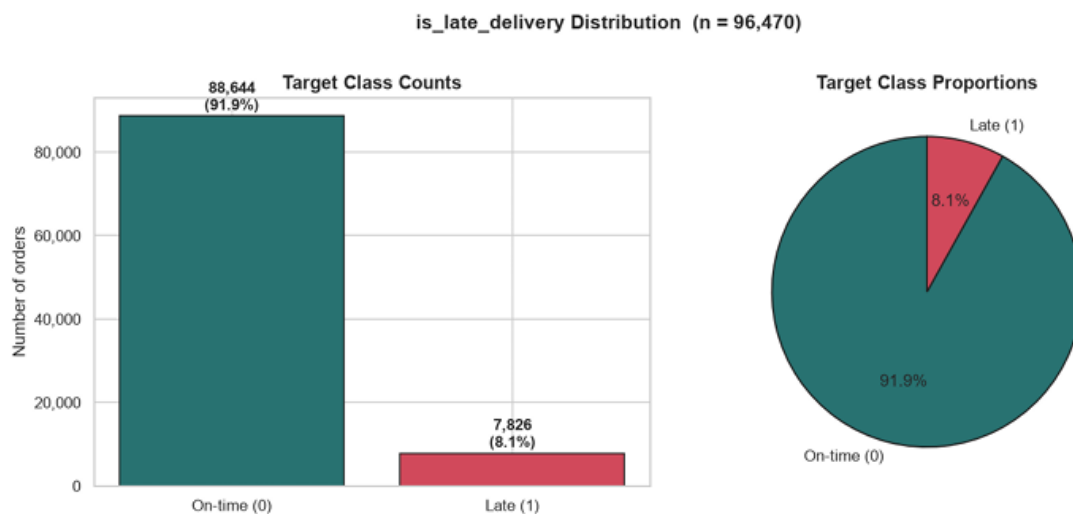


Figure 1: Labelled population: 96,470 orders, with 8.11% late under the strict timestamp rule.

Order analysis shows changes in lateness over time, including elevated rates around November 2017 and February-March 2018. The observed late rate is 9.03% in training, 5.34% in validation and 6.61% in testing. These differences motivate chronological evaluation and caution against assuming a stable future prevalence. They also mean average precision must be interpreted alongside the positive-class rate for each period.

Geographic patterns and their implications

Target-related analysis finds a Northeast late rate of 14.3299% (1,296/9,044), compared with Southeast 7.4509% and South 7.0513%. Orders with at least one cross-state seller are 9.2817% late, compared with 6.0422% where at least one seller shares the customer state. These comparisons use separately defined indicators; multi-seller orders may satisfy both an any-cross-state condition and an any-same-state condition, so the groups should not be described as universally exclusive.

Late orders have a higher mean customer-seller distance, 737.4032 km, compared with 588.4166 km for on-time orders. Exploratory distance bands range from 6.3029% late below

250 km to 13.0688% at 1,500 km or more. However, distributions overlap: the average-distance rank-biserial effect is only 0.1252 and customer-region Cramer's V is 0.0745. The observed patterns support testing geographic predictors but do not establish causation or a sufficient prediction rule. Haversine distance represents straight-line separation, not road distance or actual carrier routes.

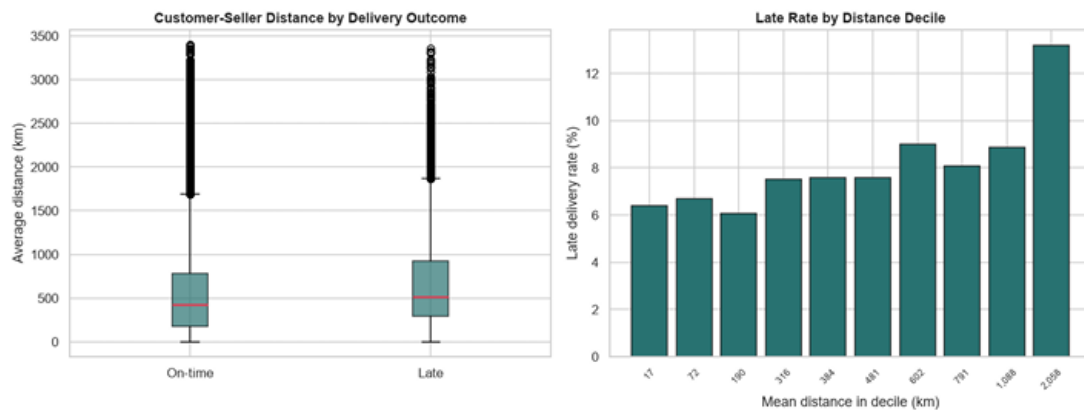


Figure 2: Distance and delivery outcome: descriptive evidence for geographic predictors, with substantial overlap.

Missingness, duplicates, skew and unusual values

Raw geolocation contains 261,831 exact duplicate records. Removing their repeated weighting before estimating ZIP-level coordinates avoids overrepresenting identical observations. Median latitude and longitude provide robust representative coordinates. Multiple valid observations for a ZIP are not all treated as duplicates: they represent the table's actual grain.

After the final merge, average/minimum/maximum distance each lack 477 values (0.494%); customer latitude and longitude each lack 265 (0.275%). Missingness is retained through explicit indicators where available, then handled during final preprocessing. Monetary values and physical product features are strongly right-skewed. Item-level IQR flags identify 8,427 price and 12,134 freight outliers; legitimate extremes are retained instead of deleted automatically. Missing categories are represented explicitly and product measurements are audited separately.

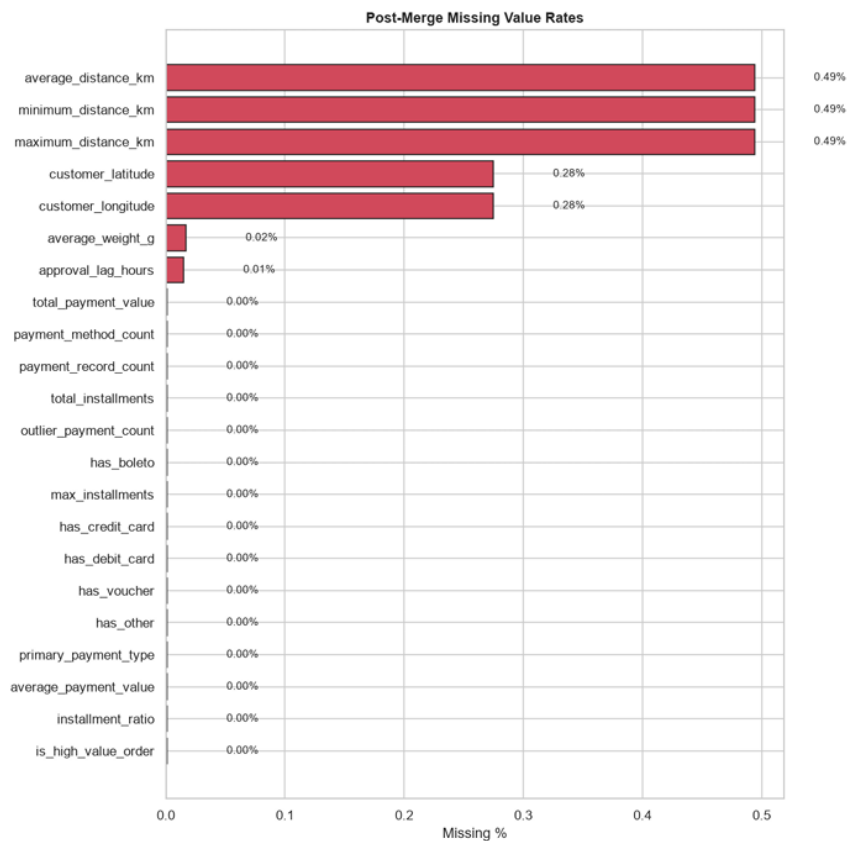


Figure 3: Post-merge missing values: measured rates support targeted imputation rather than indiscriminate row removal.

From observation to decision

The EDA supports specific choices: minority-class prevalence motivates average precision and threshold tuning; temporal variation motivates an ordered holdout; item/payment multiplicity motivates aggregation; missing geography motivates indicators and train-derived imputation; skew motivates retaining values with flags; and completed-delivery fields motivate a strict availability audit. Whole-population target-related EDA remains descriptive. Future model changes should be developed using training-period evidence so the later test set does not guide feature selection.

5. Data Cleaning and Preprocessing

Table-level preparation and validated integration

Orders are parsed into valid timestamps and filtered to the eligible target population. Customer/seller ZIP prefixes are preserved as five-character strings, maintaining leading zeros. Geographic enrichment uses deduplicated ZIP lookups and validated many-to-one relationships. Unique order-seller pairs are summarized before constructing order-level distance and agreement features. Product categories are translated and missing categories receive an explicit fallback; item prices, freight, counts, weight and volume are aggregated by order. Payments are aggregated using transaction totals, maximum/summed installments, method flags and the highest-value primary payment type.

Notebook 05 uses four validated left joins onto the labelled order spine: customer enrichment, order geography, payments and item/product features. Each intermediate output remains at 96,470 rows, and the merged table contains 61 columns. Unique/non-null keys and join cardinality checks prevent silent row multiplication. Seller and ZIP tables are loaded for audit because their information already flows through the derived order-level features.

Split and learned transformations

Partition	Orders	Late orders	Late rate	Purchase-time rule
Training	67,529	6,096	9.03%	Earliest 70%; cutoff date 15 April 2018
Validation	14,470	773	5.34%	Following 15%; cutoff date 21 June 2018
Test	14,471	957	6.61%	Most recent 15%; after second timestamp cutoff

The actual code uses purchase-timestamp quantiles, so displayed dates summarise full timestamp boundaries. Remaining continuous missing values, including distance, customer coordinates, approval lag and shipping lead time, use training-derived medians. For example, the training mean-distance median is 448.51 km and shipping-limit lead-time median is 6.01 days. Certain aggregate gaps use zero and seller-agreement gaps use false. These choices preserve rows, but zero/false can conflate unknown with absent; missingness indicators and semantics need review.

Customer state and dominant product category are encoded as their training-period target means, with the training global late rate for unseen values. Despite the notebook name 'target-frequency encoding', the calculation is a supervised target mean, not a category-frequency count. Customer region and primary payment type use training-derived one-hot categories. StandardScaler is fitted on training numerical columns and applied unchanged to later partitions. The stored modelling matrices contain 67 aligned features without missing values. However, re-running the supplied preprocessing source did not fully reproduce these stored

matrices. Consistent train-fitted imputation and versioned preprocessing artifacts are required for complete reproducibility.

Leakage controls and unresolved methodological risks

Direct leakage controls exclude actual delivery/hand-off dates, final status, reviews, identifiers and the target from predictors. Promised shipping deadlines are converted to pre-dispatch lead times, and purchase timestamps determine the chronological split before being dropped. Approval lag is valid only at the chosen post-approval prediction point.

The outer validation and test targets are not used to fit the target-mean maps or scaler. However, the maps are calculated using all outer-training labels before shuffled internal cross-validation. Consequently, an internal validation fold's labels influence its encoded inputs. Imputation/scaling also precede internal CV, and some item/payment IQR, rarity and high-value thresholds are calculated before the chronological split. These are remaining sources of internal-fold contamination or future-distribution information. They limit the claim that the entire workflow is leakage-free. A corrected experiment should fit preprocessing within each training fold and cross-fit supervised encoding with smoothing.

6. Feature Engineering and Feature Selection

Feature groups and rationale

Group	Implemented examples	Predictive rationale
Promise and time	estimated_delivery_days, approval_lag_hours, shipping_limit_lead_days, purchase month/hour/weekday/quarter	Captures promised buffer, preparation delay and changing demand
Geography	Mean/min/max distance, region, state encoding, seller-state agreement	Represents separation and cross-region fulfillment context
Order complexity	Item/product/seller/category counts, is_multi_seller	Represents handling and coordination complexity
Physical properties	Weight, volume, per-item measures, missingness counts	Represents package handling requirements and measurement quality
Commercial/payment	Price, freight, payment totals/methods/installments, outlier counts	Describes transaction structure and shipping economics
Cross-table interactions	freight_to_price_ratio, distance_x_items, freight_per_km	Tests interactions between value, distance and order size

Eight cross-table features: freight_to_price_ratio, weight_per_item, volume_per_item, is_multi_seller, is_cross_state, distance_x_items, freight_per_km and purchase_quarter. Price/distance ratios use protected denominators; distance_x_items combines mean distance and item count. Such transformations are inexpensive hypotheses about risk, not evidence that each improves generalisation. Raw shipping-limit timestamps become a duration relative to purchase, keeping the useful commitment without feeding calendar timestamps directly to the model.

Feature selection and importance

The current workflow performs rule-based exclusion of prohibited fields, identifiers and original encoded categories. It retains 67 model inputs. Random Forest importance places estimated_delivery_days first (0.0772), followed by purchase_month (0.0670) and customer_state_target_enc (0.0587). LightGBM frequently splits on estimated_delivery_days, customer coordinates, shipping lead time and approval lag. These are model-specific importance measures and should not be interpreted as causal effects.

An additional exploratory experiment compared the selected MLP settings using all 67 features and using 59 features after removing the eight cross-table interaction features.

Validation AP increased from 0.1524 to 0.1724 in this local refit. This suggests that a simpler feature set deserves further investigation. The experiment did not change the deployed model.

There are plausible redundancies: two seller-count variants, minimum/mean/maximum distance, and total versus per-item quantities. A defensible extension would compare grouped removals using chronological folds, examine permutation importance, and prefer simpler inputs when average precision remains stable. Category encodings should be evaluated using smoothed cross-fitting. No feature-selection improvement is claimed without a recorded experiment.

7. Algorithms Implemented and Their Rationale

Four complementary model families

Logistic Regression is a fast, regularised linear reference model. It estimates a linear relationship in log-odds and provides coefficients for inspection. It benefits from consistent numerical scaling but may miss nonlinear interactions unless those interactions are engineered. The search considers class weighting, although the winning configuration uses `class_weight=None`; the final report must not state that the selected Logistic Regression is balanced-weighted.

Random Forest combines bootstrapped decision trees to reduce the instability of an individual tree. It captures nonlinear thresholds and interactions with relatively little functional-form specification. Its selected `balanced_subsample` weighting increases emphasis on late orders within each bootstrap sample. A shallow Decision Tree is also implemented as a supporting reference, but the four primary compared families remain Logistic Regression, Random Forest, LightGBM and MLP.

LightGBM builds boosted trees sequentially and is a suitable structured-data comparator. Its tuned complexity, learning rate and positive-class weighting test whether a nonlinear tree ensemble better captures the order-risk relationship. The **MLP** learns nonlinear combinations through two hidden layers, using scaled inputs, L2 regularisation and early stopping. It provides a distinct neural approach and meets the requirement to compare multiple suitable algorithms.

Baselines and proposal-to-implementation changes

A most-frequent DummyClassifier provides a no-skill reference: validation ROC-AUC 0.5000, average precision 0.0534 and zero late recall. The Decision Tree reference achieves validation average precision 0.0855. Untuned versions of the four primary models provide a second comparison level. The proposal initially listed a standalone Decision Tree and XGBoost/Gradient Boosting among the intended four. The implemented comparison instead uses LightGBM and MLP alongside Logistic Regression and Random Forest, while retaining the Decision Tree as a baseline. This is a documented change of model family, not an assertion that XGBoost was implemented.

8. Model Evaluation and Comparison

The four tuned classifiers were compared on the validation set using ROC-AUC, average precision (reported as PR-AUC), and threshold-based precision, recall, and F1-score. PR-AUC was emphasized because late deliveries are the minority class; accuracy alone could look high while failing to identify late orders. The data split contained 67,529 training, 14,470 validation, and 14,471 test orders. Late-delivery rates were 9.03%, 5.34%, and 6.61%, respectively.

Model	ROC – AUC	PR – AUC	F1 at 0.5	Recall at 0.5	Precision at 0.5
MLP Neural Net	0.7766	0.1773	0.0127	0.0065	0.3486
Logistic Regression	0.7718	0.1606	0.0026	0.0013	0.1667
LightGBM	0.7408	0.1508	0.1558	0.1138	0.2456
Random Forest	0.7389	0.1505	0.0565	0.0310	0.3158

The MLP ranked first on both validation ROC-AUC and PR-AUC, so it was selected as the champion model. Its validation PR-AUC of 0.1773 was above the validation late-order prevalence of 5.34%, indicating that its ranking of late orders was better than an uninformative ranking. At the default 0.5 threshold, however, the MLP detected very few late orders. The decision threshold was therefore selected using validation data by maximizing F1-score. The chosen threshold was 0.10, giving validation F1 of 0.2551, recall of 0.4489, and precision of 0.1781.

The selected model and threshold were then evaluated on the test set. The MLP achieved ROC-AUC **0.6387**, PR-AUC **0.0983**, F1 **0.1374**, recall **0.2591**, and precision **0.0935**. It correctly identified 248 of 957 late orders, while producing 2,404 false positives. This reflects a practical trade-off: the model identifies some late orders, but most orders flagged as late are actually on time. Test performance was lower than validation performance, so the validation results should not be treated as a reliable estimate of deployment performance. The drop is consistent with possible temporal distribution shift, but further analysis would be needed to establish its cause.

Overall, the MLP was the strongest candidate by the project’s primary validation metric, but its test results show that late-delivery prediction remains challenging. Its low test precision means the system should be treated as a prioritization aid rather than a definitive late-delivery decision. Future evaluation should consider whether the operating threshold better matches the cost of missed late orders versus unnecessary alerts.

9. Hyperparameter Tuning and Optimization

Hyperparameters were selected using cross-validation on the training set. Average precision (PR-AUC) was used as the search scoring metric because late deliveries are the minority class. The validation set was kept separate for model comparison, and the test set was reserved for final evaluation.

Model	Search Method and Best Settings	Best Cross-validation PR-AUC
Logistic Regression	5-fold stratified grid search C=0.1, class_weight=None	0.2204
Random Forest	5-fold stratified randomized search over 10 combinations n_estimators=300, max_depth=16, min_samples_split=5, max_features=sqrt, class_weight-balanced_subsample	0.2909
LightGBM	5-fold stratified randomized search over 12 combinations n_estimators=300, learning_rate=0.05, max_depth=8, num_leaves=63, subsample=1.0, colsample_bytree=0.7, scale_pos_weight=5.04	0.3203
MLP Neural Net	3-fold stratified grid search, hidden layers (64,32), alpha=0.001, activation=relu	0.2491

The Logistic Regression search tested four values for C and two options for class-weight. The Random Forest search varied the number of trees, depth, split size, feature sampling and class weighting. The LightGBM search explored boosting rounds, learning rate, tree complexity, row and column sampling, and minority-class weighting. The MLP search varied network architecture and L2 regularization; early stopping was enabled, and the selected network trained for 16 iterations.

The search results are internal cross-validation estimates, not final model performance. In particular, the LightGBM's best cross-validation PR-AUC (0.3203) was substantially higher than its validation PR-AUC (0.1508), and the MLP's cross-validation PR-AUC (0.2491) was higher than its validation PR-AUC (0.1773). The folds were shuffled and stratified, while the project's final evaluation uses time-separated data; this difference may make cross-validation less representative of performance on later orders. Also, MLP used three folds while the other models used five, so the cross-validation scores should not be treated as a direct model ranking. The common validation comparison is the more appropriate basis for model selection.

Decision-threshold tuning was performed separately from hyperparameter search: the MLP threshold was selected on validation data by maximizing F1-score, resulting in a threshold of 0.10. That threshold was then applied to the held-out test set.

Baseline versus saved tuned-model comparison (validation average precision)

Model	New baseline AP	Saved tuned-model AP
Logistic Regression	0.1575	0.1608
Random Forest	0.1290	0.1473
LightGBM	0.1417	0.1508
MLP	0.1578	0.1768

The local audit compared newly fitted baseline models with the saved tuned artifacts. Each saved tuned model achieved higher validation average precision than its corresponding new baseline. The saved MLP achieved the highest AP of 0.1768. This is a local artifact evaluation; the original full hyperparameter searches were not repeated.

10. Final Model Selection and Justification

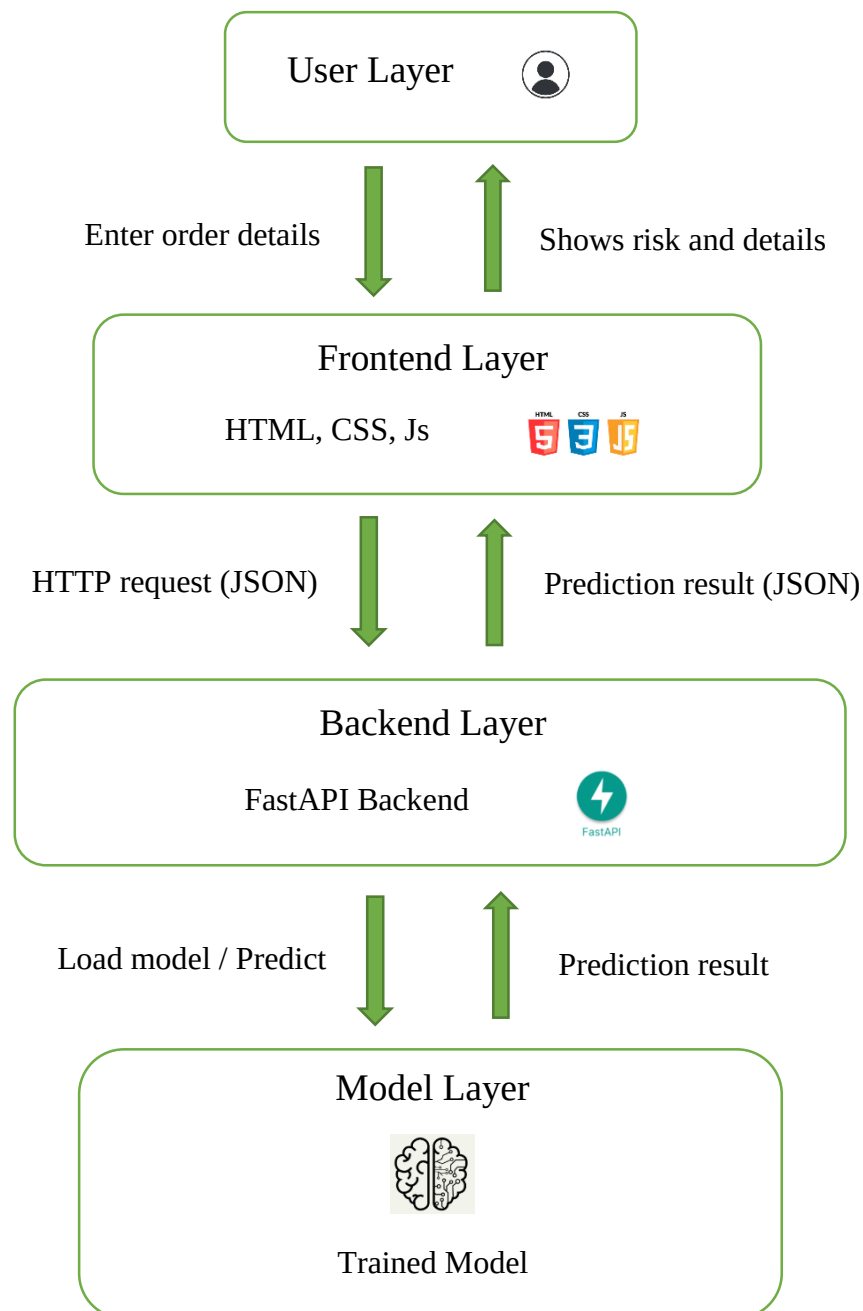
The final model selected was the MLP Neural Network. Since the minority class is late deliveries, and the goal is to find potentially late orders, we have selected validation PR-AUC for the project. The MLP achieved the best PR-AUC (0.1773) and ROC-AUC (0.7766) among the three other candidates in the validation set. Logistic regression is second-best in these measures. The best F1-score and recall at the default threshold 0.5 is obtained by lightgbm. Because PR-AUC was the pre-defined primary selection metric, the MLP was chosen.

The classification threshold was tuned separately on the validation set by maximizing F1-score. A threshold of 0.10 improved validation F1 to 0.2551, with 44.89% recall and 17.81% precision. This lower threshold favors identifying more late orders, at the cost of generating more false alerts.

On the held-out test set, using the selected model and threshold, the MLP achieved PR-AUC 0.0983, ROC-AUC 0.6387, F1 0.1374, recall 25.91%, and precision 9.35%. It detected 248 of 957 late orders. The reduction from validation performance indicates that results on later orders were weaker than validation results; therefore, the model should be regarded as a screening aid rather than a reliable final determination of delivery risk. Its low precision also means that any operational use should account for the cost of investigating false alerts.

The selection is based on the project's stated validation criterion, not on the highest cross-validation score: LightGBM had the highest recorded cross-validation PR-AUC, but the MLP had the highest validation PR-AUC. The final test results were used to assess the selected model, not to choose among models.

11. System Architecture and Implementation



System Architecture

The proposed system follows a modular pipeline architecture composed of three main layers: data preparation, model training, and real-time prediction. In the first layer, raw e-commerce datasets from Olist are processed using a structured data-mining workflow. The raw transactional data, including order records, customer information, seller data, product details, and payment information, are cleaned, merged, and transformed into a consistent analytical

dataset. Feature engineering is carefully performed to derive variables such as geographic distance, shipping characteristics, order timing, and payment behavior while avoiding data leakage from post-delivery information. The outcome of this stage is a reliable feature matrix suitable for supervised learning.

The second layer is the model development stage. The cleaned dataset is split into training, validation, and testing subsets, and multiple machine learning algorithms are evaluated to identify the most accurate classifier for predicting late deliveries. The selected model is then serialized along with the required preprocessing artifacts, including the scaler and feature metadata, for use during inference. The backend constructs 67 features in the expected order and applies the saved scaler to 44 numeric features. The system successfully generates predictions through the web interface. Exact equivalence between the training and serving transformations requires further verification.

The third layer is the operational prediction system, which exposes the trained model through a lightweight web-based interface. The backend receives a new order request containing characteristics such as product category, delivery timing, ZIP codes, price, freight value, and package dimensions. These inputs are converted into the same feature structure used during training using a dedicated feature engineering module. The model then generates a probability score indicating the likelihood of late delivery, and the result is classified into Low, Medium, or High risk. The frontend presents this information to the user in an intuitive dashboard, making the model accessible to non-technical stakeholders.

In summary, the system architecture is based on a clear separation between offline analytics and online prediction. This design improves maintainability by keeping model development separate from the prediction service.

Implementation

The implementation was carried out using a Python-based machine learning pipeline integrated with a web application for prediction. The project begins with data preparation notebooks that perform exploratory data analysis, preprocessing, feature construction, and final dataset preparation. These notebooks ensure the data quality and create the analytical foundations required for model training. Since the project deals with real-world e-commerce data, careful attention was given to missing values, inconsistencies, and leakage prevention so that only pre-dispatch information is used in the prediction process.

The machine learning stage uses a supervised classification approach, where the target variable is defined as whether the order is delivered after the estimated delivery date. Different models are tested and compared, and the best-performing model is selected based on predictive accuracy and classification quality. Once the model is trained, it is stored as a reusable artifact, along with the scaler and feature list, so the endpoint can make predictions without re-running the training workflow.

The deployment layer is implemented as a REST API using FastAPI, which is well-suited for lightweight prediction services because of its speed, simplicity, and automatic validation features. The backend validates incoming requests, transforms input values into engineered features, invokes the trained model, and returns the predicted probability and risk category.

The application also includes health-check endpoints and structured error handling to maintain reliability during use.

A simple frontend interface is included to enable users to enter order details and view prediction results in real time. The front end communicates with the backend through HTTP requests and displays the risk score, threshold information, and classification summary. This creates a complete end-to-end system that can be used both as a proof of concept and as a practical decision-support tool for operations teams.

Overall, the implementation demonstrates a strong integration of data mining, software engineering, and user-facing deployment. It combines a disciplined analytical workflow with a functional prediction service, making the system both technically robust and practically useful for real-world late-delivery risk management.

12. Backend and Frontend Development

Backend Development

The backend was developed as a REST API using FastAPI. It provides health-check endpoints (/ and /health) and a prediction endpoint (POST /predict). The health endpoints allow the frontend to check whether the service is available, while the prediction endpoint accepts order details as JSON.

Request and response structures are defined using Pydantic schemas. The request schema covers order timing, customer and seller ZIP prefixes, product category, price, freight, package dimensions, and payment information. It also supports optional values such as item count, seller count, and payment approval lag. Input constraints are applied to relevant fields, including ranges for ZIP prefixes and non-negative values for freight and package weight.

When a valid request reaches the prediction endpoint, the backend passes the input to a feature-building module. This module derives 67 model features from the supplied values and local lookup files, then applies the fitted scaler to 44 numeric features in the order expected by the model. The predictor uses the saved model to calculate the probability of late delivery and returns the probability, binary predicted class, risk label, thresholds used, and number of computed features. Risk labels are assigned as Low, Medium, or High using configured probability cutoffs; the binary class uses the decision threshold stored with the model when available.

Frontend Development

The frontend was implemented using HTML, CSS, and vanilla JavaScript. It provides a web interface for entering order information and viewing the prediction. The interface includes navigation, a prediction form, a theme toggle, a sample-order option, and a result panel.

Before sending a request, JavaScript performs client-side checks, including required-field checks, date ordering, and ZIP-prefix and numeric-value validation. It then sends the form data as a JSON request to the backend's /predict endpoint. While the request is being processed, the interface displays a loading state. On success, it presents the predicted risk category, late-delivery probability, binary class, and threshold details. If the request fails, an error message is shown to the user.

The frontend also checks the backend's health endpoint to indicate whether the API is online. It is configured to try the local API on ports 8002, 8001, and 8000, allowing it to connect to the available development server.

End-to-End Prediction Flow

The user enters the order information in the web form. The frontend validates the entries and sends them to the FastAPI backend. The backend validates the request, constructs and scales the model features, and obtains a prediction from the saved model. It returns the prediction as JSON, which the frontend displays as a probability and risk category. This separation allows the model-inference logic to be maintained independently from the user interface.

13. System Testing and Results

The Olist Late Delivery Risk Prediction system was tested on 5 October 2026 to assess whether operations and logistics users could submit order information and receive a late-delivery risk assessment before dispatch. Testing covered the running backend, valid and invalid inputs, feature-vector compatibility, risk classification and repeated prediction requests. The frontend HTML, CSS and JavaScript were checked for successful delivery, and the frontend integration code was reviewed.

Testing approach

The frontend ran at <http://127.0.0.1:8080> and the FastAPI backend at <http://127.0.0.1:8002>. Requests were submitted directly to the running `/predict` endpoint. Twenty prediction test cases were executed. A successful valid request was expected to return HTTP 200 with a probability, risk label and binary class. Missing or invalid values were expected to return HTTP 422. Unknown ZIP prefixes and non-empty unseen categories were tested separately as supported fallback cases. These checks evaluated system behaviour rather than the correctness of an individual real-world delivery outcome.

Functional and input-validation results

Test	Expected HTTP	Actual HTTP	Result
Valid demo order	200	200	Pass
Optional defaults	200	200	Pass
Missing price	422	422	Pass
Zero price	422	422	Pass
Negative freight	422	422	Pass
ZIP below range	422	422	Pass
ZIP above range	422	422	Pass
Invalid numeric type	422	422	Pass
Zero dimension	422	422	Pass
Installments above limit	422	422	Pass
Zero item count	422	422	Pass
Malformed date	422	500	Gap identified
Delivery before purchase	422	200	Gap identified
Blank category	422	200	Gap identified
Unknown payment method	422	200	Gap identified
Zero weight	200	200	Pass
Unknown ZIP fallback	200	200	Pass

Unknown category fallback	200	200	Pass
Multiple items and sellers	200	200	Pass
Tighter delivery window	200	200	Pass

Testing confirmed successful predictions for valid orders and correct rejection of most invalid numeric inputs. Sixteen of twenty API cases passed. Date-format, date-order, blank-category and unsupported-payment validation require improvement. These checks assess application behaviour rather than model prediction accuracy.

Prediction and integration results

A sample order used a purchase timestamp of 15 July 2018 at 14:30 and estimated delivery of 10 August 2018, customer ZIP prefix 1001, seller ZIP prefix 4101 and product category `bed_bath_table`. Its unit price was R\$120, freight R\$25, weight 1,500 g and package dimensions 30 x 15 x 20 cm. Payment was by credit card with three installments. The system returned a late-delivery probability of 0.0139 (1.39%), a Low risk label and predicted class 0 (On-Time). This demonstrates a successful request-to-prediction path; the sample did not have a verified delivery outcome, so it cannot establish prediction correctness. The feature builder produced a finite vector containing 67 features in the expected saved column order. The model expected 67 inputs, and the fitted scaler processed 44 numeric features. This confirms feature dimension and ordering compatibility for the tested sample; exact equivalence between every training and serving transformation was not established.

Risk-boundary checks

Probability	Observed risk label
0.0000	Low
0.1000	Low
0.2999	Low
0.3000	Medium
0.5499	Medium
0.5500	High
1.0000	High

The observed risk boundaries were Low below 30%, Medium from 30% to below 55%, and High at or above 55%. Risk categories and the binary alert use different thresholds. Scores from 0.10 to below 0.30 produce a Late alert with a Low risk label. The interface recommendation should prompt human review whenever a Late alert is raised. Frontend code review confirmed that form values are sent as a JSON POST to the backend and that the returned probability, risk label, class and recommendation are displayed. Loading and error-handling logic are present. Browser checks verified demo-order submission, prediction

display, reversed-date validation, theme switching and no horizontal overflow at a 390-pixel viewport. These checks demonstrate the tested workflow; full accessibility and stakeholder acceptance remain future work.

Prediction reproduction check

Measure	Local test result
Number of orders	14,471
Actual late orders	957
ROC-AUC	0.6389
Average precision	0.0982
Late-class precision	0.0929
Late-class recall	0.2571
Late-class F1	0.1364
Confusion matrix	TN = 11,111; FP = 2,403; FN = 711; TP = 246

The local check detected 246 late orders, compared with 248 in the recorded notebook experiment. The saved-model and runtime library versions differed, so exact reproduction remains to be verified.

Overall system testing result

The prototype successfully served its interface and generated structured late-delivery predictions through the backend. Core numeric validation, sample feature compatibility and deterministic repeated responses were demonstrated. Four input-validation cases did not meet the acceptance criteria, and the risk-label interpretation, complex-order approximations and reproduction discrepancy remain limitations. The system is suitable for an academic demonstration of an early-warning workflow, but the tests do not establish production readiness or reliable automated intervention. Its output should support human review rather than be treated as a guaranteed delivery outcome.

14. Limitations and Possible Future Improvements

This project demonstrates how machine learning can support early identification of late-delivery risk, but several limitations should be considered when interpreting its results and applying it in practice.

Limitations

1. **Historical and geographically limited data**

The model is based on Olist's Brazilian e-commerce data from 2016–2018. Customer behavior, sellers, logistics networks, and delivery practices may have changed since then. The results may not generalize to other countries, platforms, or current operations without additional validation.

2. **Further validation is still needed**

Held-out chronological testing was completed. Further work should validate the system on newer external data, improve input handling and ensure that preprocessing and model artifacts are reproduced consistently.

3. **Limited order detail at prediction time**

The application requests a compact set of order-level inputs and derives additional features from them. Some derived values rely on assumptions—for example, treating the order as having one product or category in certain feature calculations. This may not fully represent complex orders containing multiple products, categories, or sellers. The feature-building assumptions are implemented in `feature_builder.py`.

4. **Geographic estimates are approximate**

ZIP-prefix lookups provide representative coordinates rather than exact addresses. Distance is calculated as straight-line geographic distance, not actual road distance, travel time, or carrier route. When a ZIP prefix is not found, the current implementation uses fallback coordinates; this can make the resulting distance less representative of the order's actual geography.

5. **Probability and risk labels may need operational calibration**

Low, Medium, and High labels are assigned using fixed probability cutoffs, while the binary prediction uses the model's decision threshold. These thresholds may not reflect the costs of late deliveries, the resources available for intervention, or the organization's tolerance for missed high-risk orders.

6. **Prototype-level application operation**

The current application is designed as a lightweight prediction prototype. It uses local model and lookup files, and the implementation does not show user authentication, persistent prediction logging, production deployment monitoring, or automated model updates. These capabilities would be needed for a larger operational deployment.

Possible Future Improvements

- **Evaluate on newer and independent data:** Test the selected model on a later time period and, where possible, on data from additional marketplaces or regions. Report test-set metrics separately from validation results.
- **Represent multi-item orders more accurately:** Extend the input interface and feature builder to process multiple products, categories, and sellers, and calculate order-level aggregates directly from the complete item list.
- **Improve geographic features:** Use more precise location data and, if available, route distance or estimated transit time. Track missing or approximate location values explicitly and avoid allowing fallback coordinates to imply precise distances.
- **Calibrate the model and decision thresholds:** Assess probability calibration and select intervention thresholds using operational costs, such as the cost of a missed late order versus the cost of unnecessary follow-up.
- **Add explainability:** Show the main factors contributing to an individual risk estimate, where supported by the selected model, so users can better understand and assess predictions.
- **Strengthen the application for deployment:** Add automated API and end-to-end tests, controlled access, production-appropriate CORS settings, structured prediction logging, and monitoring for service failures and model performance changes.
- **Establish a model maintenance process:** Monitor prediction and outcome data over time, then periodically review whether retraining or feature updates are needed.

These improvements would help determine whether the model generalizes beyond the historical dataset, make its predictions more useful for complex orders, and prepare the prototype for more reliable operational use.

15. Individual/Group Contributions, Where Required

Registration Number	Contribution
IT23809710 Lokuhewage G L N D	✓ Explored and prepared the order data; defined the late-delivery target; documented target-related data quality and leakage considerations. ✓ Logistic Regression model development ✓ Prepare the model for inference ✓ Final Report - Introduction and problem definition, Assigned scenario and user/stakeholder requirements, Dataset identification, source, citation, and validation
IT23858770 Wathsara W H H	✓ Explored and prepared customer, seller, and geolocation data; generated geographic enrichments and order-level geographic features; documented matching coverage and geographic-data limitations. ✓ Random forest model development ✓ Build the prediction service ✓ Final Report - Data understanding and EDA, Data cleaning and preprocessing, Feature engineering and feature selection, Individual/group contributions, where required
IT23559004 Irunika W N	✓ Explored and prepared product and order-item data; developed item- and product-related features, including product-category information. ✓ LightGBM model development ✓ Build the user-interface ✓ Final Report - Algorithms implemented and their rationale, Model evaluation and comparison, Hyperparameter tuning and optimization, Final model selection and justification
IT23720824 Dulshini W G S	✓ Explored and prepared payment data; integrated the prepared datasets; created the final modelling dataset and data splits. ✓ MLP Neural Net model development ✓ Verify the complete system ✓ Final Report - System architecture and implementation, Backend and frontend development, System testing and results, Limitations and possible future improvements